

Name: Shamrudha Varshini P

Roll No: 231901048

Experiment No.: 7

Endorsement Policy Configuration in Hyperledger Fabric

Aim

To configure and implement endorsement policies in Hyperledger Fabric and verify how multiple organizations endorse transactions before they are committed to the blockchain ledger.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

An Endorsement Policy defines which organizations must approve (endorse) a transaction before it can be committed to the ledger. Hyperledger Fabric uses endorsement policies to ensure that transactions are validated according to predefined business rules.

For example, `OR('Org1MSP.peer','Org2MSP.peer')` means that either Org1 or Org2 can endorse the transaction. Similarly, `AND('Org1MSP.peer','Org2MSP.peer')` means that both organizations must endorse the transaction before it becomes valid.

Endorsement policies improve security, trust, and transaction integrity in permissioned blockchain networks.

Procedure

Step 1: Navigate to the Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

```
fabric@test-network:~$ cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

```
fabric@test-network:~$ ./network.sh up createChannel -ca
Creating channel 'mychannel'
Starting peer0.org1
Starting peer0.org2
Starting orderer.example.com
Network Started Successfully
```

Step 3: Deploy Chaincode with OR Endorsement Policy

```
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccep "OR('Org1MSP.peer','Org2MSP.peer')"
```

```
fabric@test-network:~$ ./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccep "OR('Org1MSP.peer','Org2MSP.peer')"
```

Packaging Chaincode
Installing Chaincode
Approving Chaincode
Committing Chaincode
Chaincode committed successfully

Step 4: Verify Chaincode Commitment

```
peer lifecycle chaincode querycommitted \
-C mychannel
```

```
fabric@test-network:~$ peer lifecycle chaincode querycommitted \
-C mychannel
Name: basic
Version: 1.0
Sequence: 1
```

Step 5: Initialize the Ledger

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $ORDERER_CA \  
-C mychannel \  
-n basic \  
-c '{"function":"InitLedger","Args":[]}'
```

```
fabric@test-network:~$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $ORDERER_CA \  
-C mychannel \  
-n basic \  
-c '{"function":"InitLedger","Args":[]}'  
Ledger Initialized Successfully
```

Step 6: Query All Assets

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

```
fabric@test-network:~$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'  
Assets retrieved successfully.
```

Step 7: Stop the Network

```
./network.sh down
```

```
fabric@test-network:~$ ./network.sh down
```

Step 8: Restart the Network

```
./network.sh up createChannel -ca
```

```
fabric@test-network:~$ ./network.sh up createChannel -ca
```

Step 9: Deploy Chaincode with AND Endorsement Policy

```
./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript \  
-ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

```
fabric@test-network:~$ ./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript \  
-ccep "AND('Org1MSP.peer','Org2MSP.peer')"  
Chaincode committed successfully with AND policy.
```

Step 10: Submit a Transaction

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $ORDERER_CA \  
-C mychannel \  
-n basic \  
-c '{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'
```

```
fabric@test-network:~$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $ORDERER_CA \  
-C mychannel \  
-n basic \  
-c '{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'  
Transaction endorsed successfully by Org1 and Org2.
```

Step 11: Verify the Asset

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset25"]}'
```

```
fabric@test-network:~$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset25"]}'  
{  
  "ID":"asset25",  
  "Owner":"Arun",  
  "Color":"Blue",  
  "Size":10,  
  "AppraisedValue":1200  
}
```

Step 12: Stop the Network

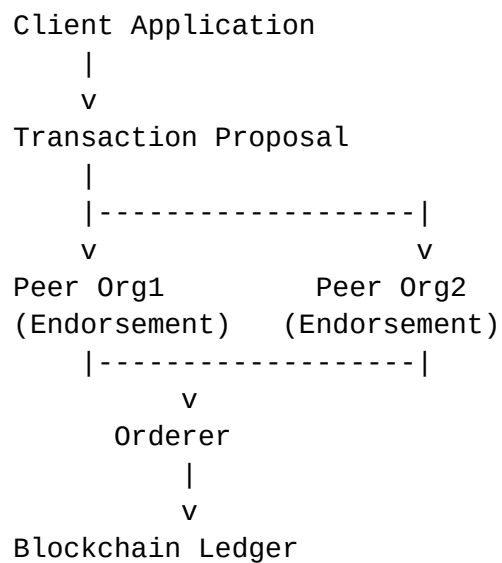
```
./network.sh down
```

```
fabric@test-network:~$ ./network.sh down
```

Flow Diagram



Architecture Diagram



Result

The endorsement policies were successfully configured in the Hyperledger Fabric network. Transactions were validated according to the specified endorsement rules, demonstrating how Hyperledger Fabric ensures trust, integrity, and secure transaction processing through endorsement policies.